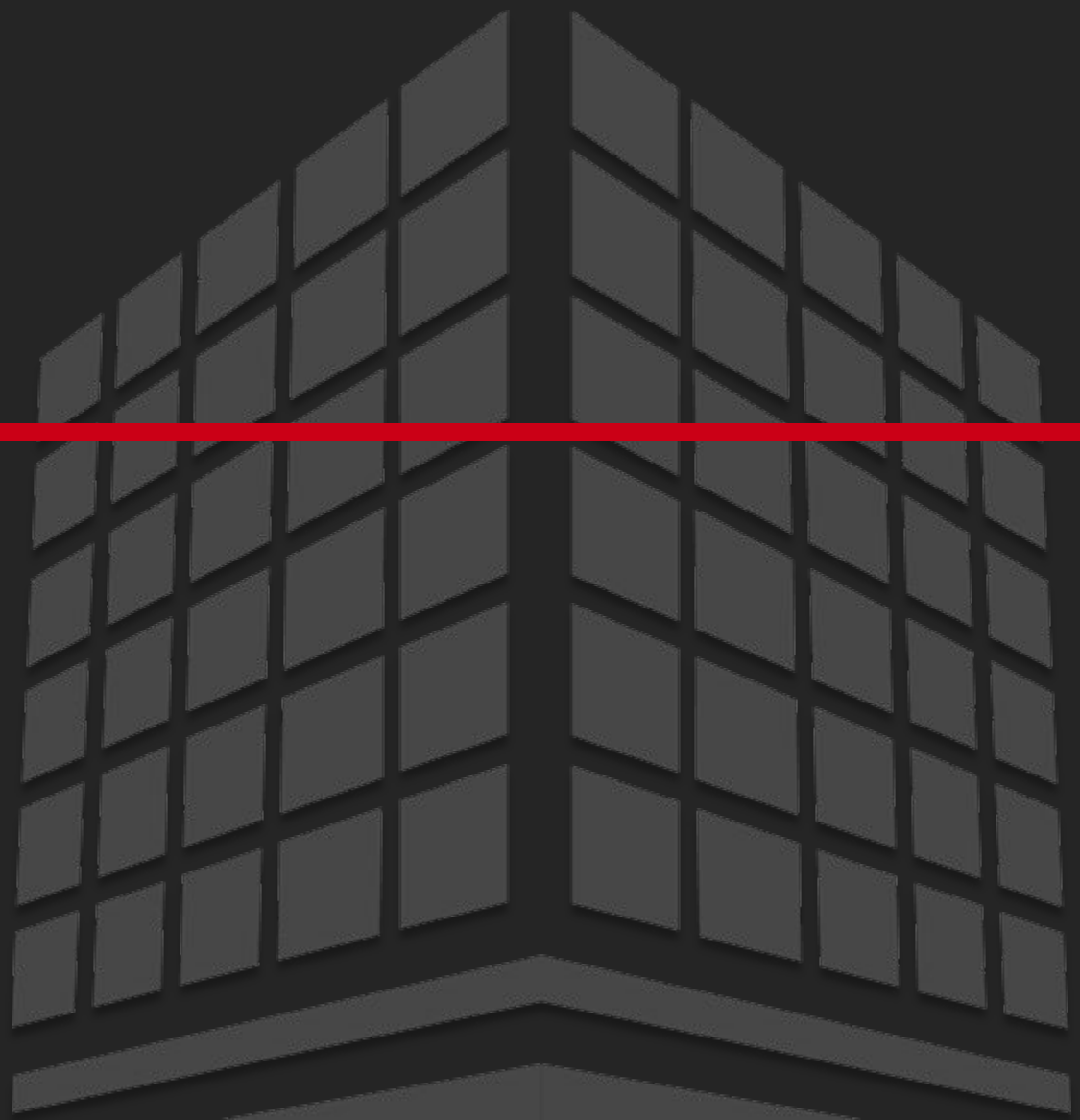
Reversing



Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

¿Que es reversing?



tu sabiendo reversing

Reversing (Reverse Engineering)

Es el proceso de entender cómo está hecho y cómo funciona un sistema ya terminado

Usos prácticos:

- entender el funcionamiento de un malware
- conseguir free alguna licencia de producto
- encontrar backdoors en sistemas

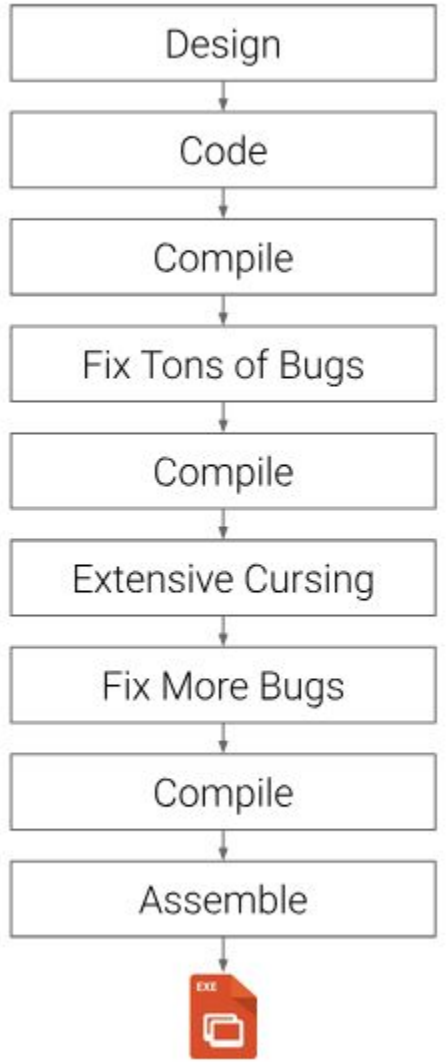
Universidad Rey Juan Carlos

Construyendo un programa

Cuando construimos un programa el proceso es el siguiente:

1. Decides qué quieres programar
2. Programas
3. Compilas
4. Ejecutas

En cada paso se pierde información!

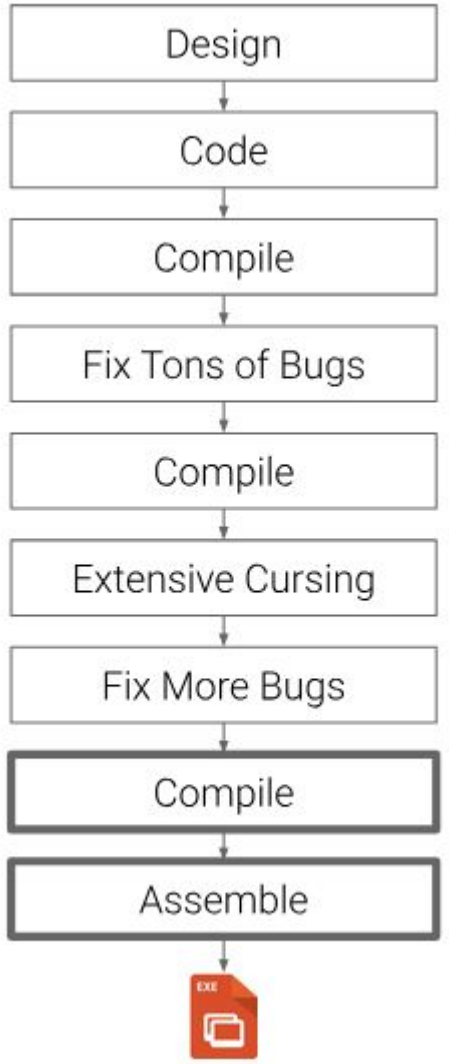


Universidad Rey Juan Carlos

¿Qué se pierde al compilar?

¿Qué información se pierde al compilar un programa?

- Comentarios
- Nombres de variables
- Nombres de funciones
- Estructuras de datos (structs, clases, …)
- *En algunos casos algoritmos enteros (optimización)*

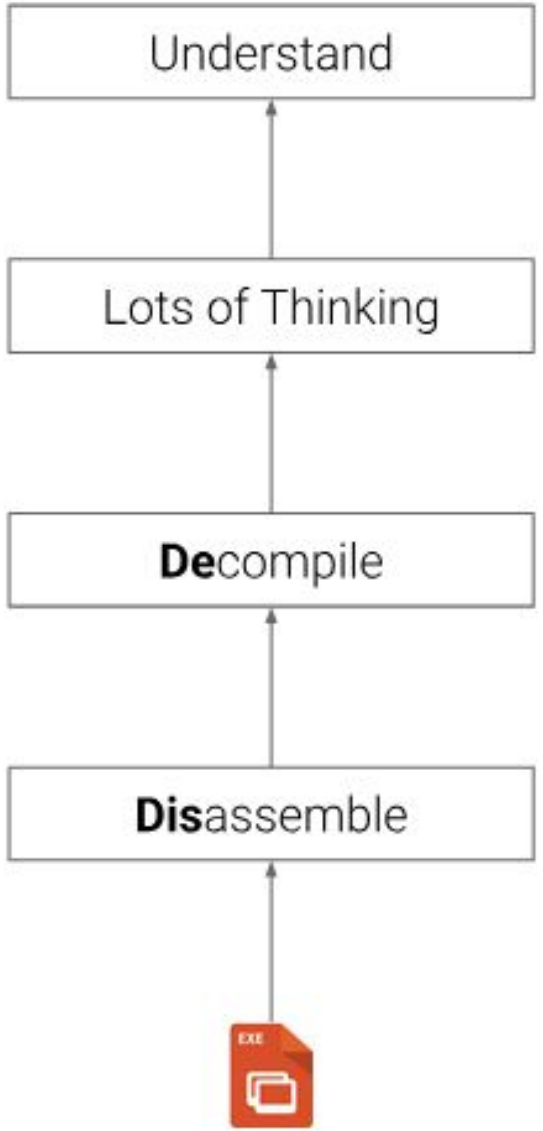


Universidad Rey Juan Carlos

El proceso de hacer reversing

El objetivo haciendo reversing es **entender el diseño del programa a partir del compilado**

Normalmente, un reverser hace uso de herramientas especializadas para construir un modelo mental del software objetivo



Universidad Rey Juan Carlos

¿Qué es compilar?



resulta que tu compu del gobierno no es capaz de entender tal cual un:

```
int iVar1;
char input [15];
char activacion [15];
```

Entonces si no entiende tal cual esto, cómo hacemos que lo entienda

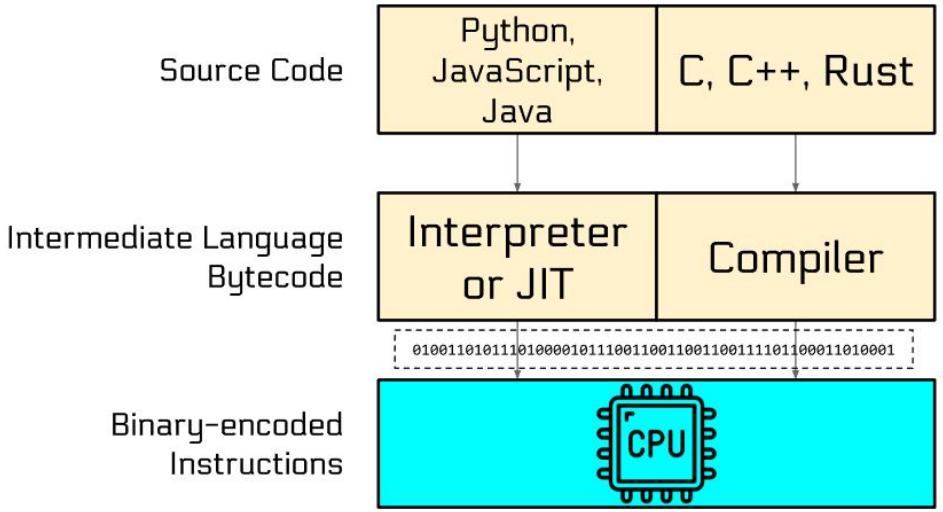
Universidad Rey Juan Carlos

Todos los caminos llevan a Assembly

Como ya sabréis (o deberíais saber) para ejecutar código en C hay que pasarlo por un **compilador.**

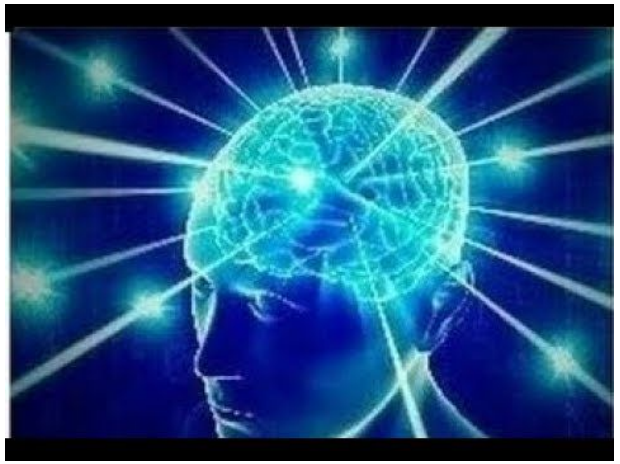
El compilador se encarga de pasar nuestras líneas de código de alto nivel a instrucciones en assembly **QUE NUESTRO PROCESADOR SÍ ENTIENDE.**

Esto pasa de una forma u otra para **TODOS LOS PROGRAMAS.**



Universidad Rey Juan Carlos

¿Cómo se pasa de un código fuente a assembly?



Un compilador traduce un lenguaje a assembly para que el procesador lo pueda entender

Código en *C*

```c
int main(){
  int a = 5;
  int b = 3;
  int c = a+b;
  return c;
}
```

Código en *Assembly x86/x64*

```
push    rbp
mov     rbp, rsp
mov     DWORD PTR [rbp-4], 5
mov     DWORD PTR [rbp-8], 3
mov     edx, DWORD PTR [rbp-4]
mov     eax, DWORD PTR [rbp-8]
add     eax, edx
mov     DWORD PTR [rbp-12], eax
mov     eax, DWORD PTR [rbp-12]
pop     rbp
ret
```

Universidad Rey Juan Carlos

¿Y al revés?

¿Se puede pasar del binario compilado al código original?



NO

*pero se puede conseguir algo bastante parecido ;)



Análisis estático



Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

Decompiladores

Un decompilador es un programa que a partir de un binario te da la mejor predicción del código original

EJEMPLOS:



IDA



Ghidra



Binary Ninja

Universidad Rey Juan Carlos

Ghidra I



¿Cómo decompilo un binario?

Ghidra es la herramienta que utilizaremos

Su función es coger el ejecutable y mostrarnos su código en ensamblador

También nos traduce el ensamblador a un pseudocódigo C para que nosotros lo podamos leer.

Ahora veremos un ejemplo guiado de cómo usar ghidra

Universidad Rey Juan Carlos

Ejemplo guia de reversing (photoslop)



tu

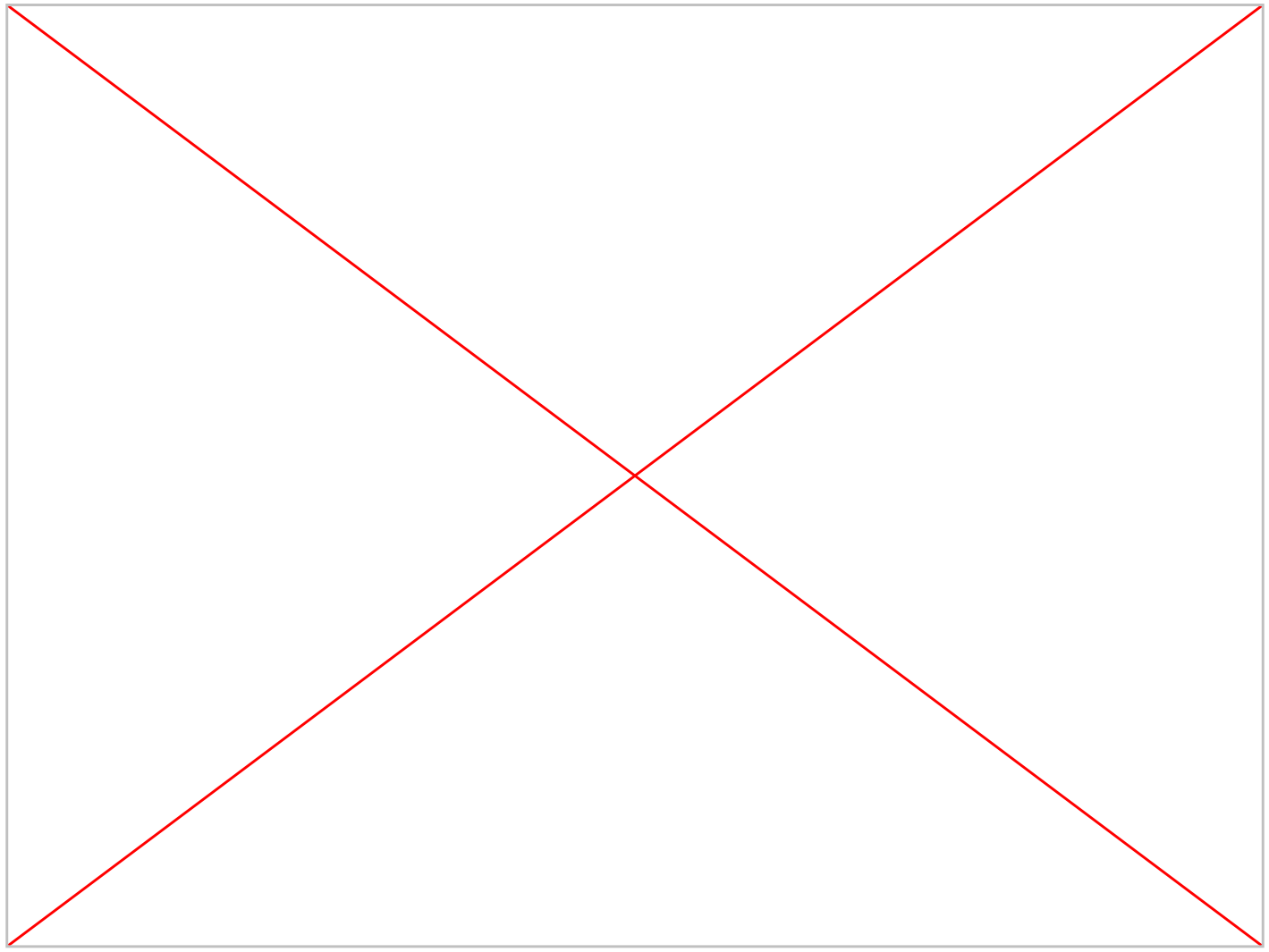


tu habiendo conseguido photoslop

GRATIS

Universidad Rey Juan Carlos

Photoslop crack pt1



Universidad Rey Juan Carlos

Photoslop crack for real



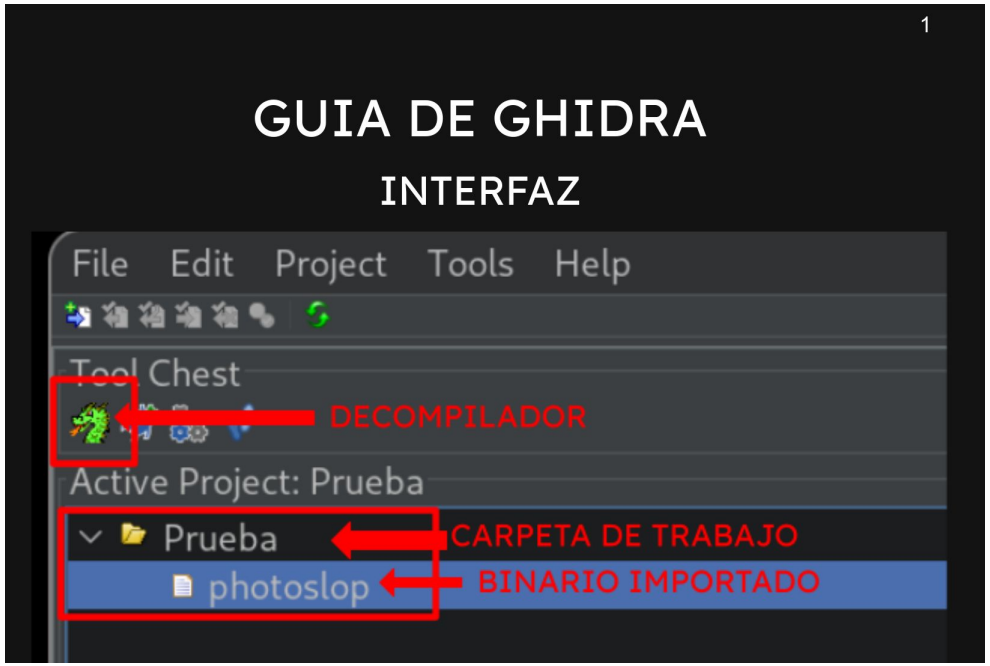
(para los que no han estado en clase / se os ha olvidado)

Universidad Rey Juan Carlos

Ghidra II

Os adjunto una cheatsheet de ghidra de diferentes comandos útiles (está en la web)

https://docs.google.com/document/d/1GBogJ599b5UOGT3F_RzV8XdS4BUXvXk-xCBJjpB37Ag/edit?usp=sharing



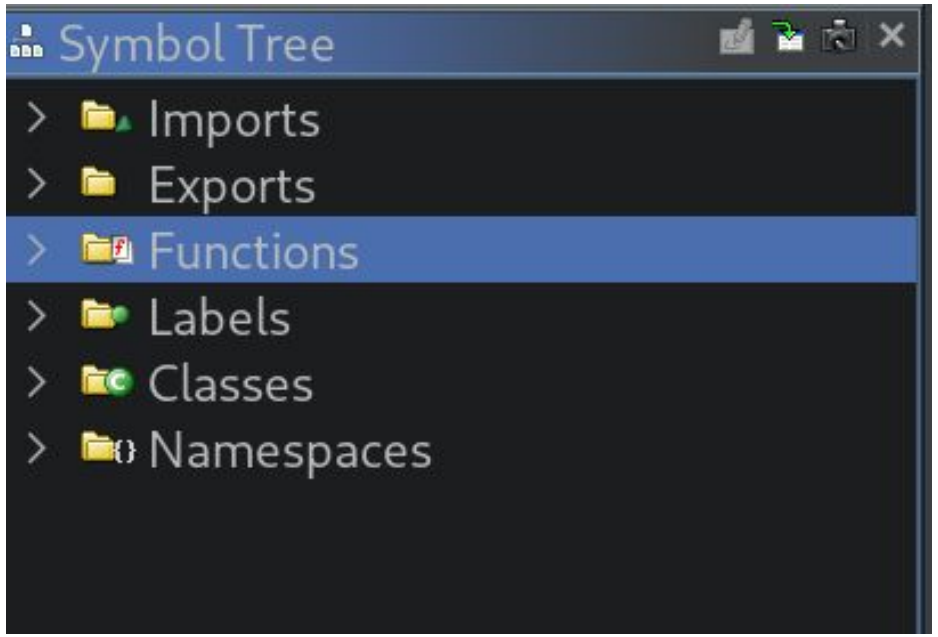
vosotros gracias a la cheatsheet

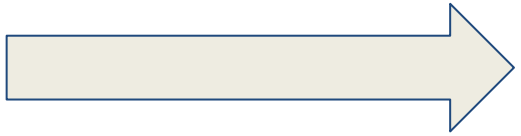


Universidad Rey Juan Carlos

Símbolos en ghidra

un símbolo es una etiqueta de texto
que conecta un nombre con una dirección de memoria







Es como el índice de un libro

Universidad Rey Juan Carlos

Un binario stripped



Un binario stripped es que te quitan los símbolos y metadatos.
(a si que estas como en la foto)

Para ello tenemos trucos
(ver cheatsheet)

Universidad Rey Juan Carlos

No entiendo el código :(

Hay situaciones en las que el pseudo-c es **MUY** complicado de entender

```
  puVar5 = auStack_30;
  while (puVar5 = puVar5 + 1, auStack_30 + 4 != puVar5) {
    *puVar5 = ((*puVar5 ^ 0x55aa55aa) >> 0x19 | (*puVar5 ^ 0x55aa55aa) << 7) + 0x1337c0de;
  }
  auStack_30[0] =
       ((auStack_30[0] ^ 0x55aa55aa) >> 0x19 | (auStack_30[0] ^ 0x55aa55aa) << 7) + 0x1337c0d
       auStack_30[1];
  iVar7 = iVar7 + -1;
  auStack_30[1] = auStack_30[1] ^ auStack_30[2];
  auStack_30[2] = auStack_30[2] + auStack_30[3];
  auStack_30[3] = auStack_30[3] ^ auStack_30[0];
} while (iVar7 != 0);
puVar5 = auStack_30 + 4;
auStack_30[4] = 0x7087827c;
apuStack_1c[0] = (uint32_t)&DAT_5da0dd38;
apuStack_1c[1] = 0xed862143;
apuStack_1c[2] = 0xe7b8ea13;
if (auStack_30[0] == 0x7087827c) {
```



Universidad Rey Juan Carlos

¿Qué hacemos?

Cuando tenemos esta situación podemos proceder de 3 formas:

- Parchear el Binario
  - Cambiar el comportamiento
  - Saltarnos un Check
  - Cambiar el valor inicial de una variable

- Análisis Dinámico
  - Ver el valor de una variable en mitad de la ejecución
  - Cambiar el valor de una variable en mitad de la ejecución
  - Inspeccionar la memoria

- Seguir intentando entender el pseudo-C *(a veces es necesario)*



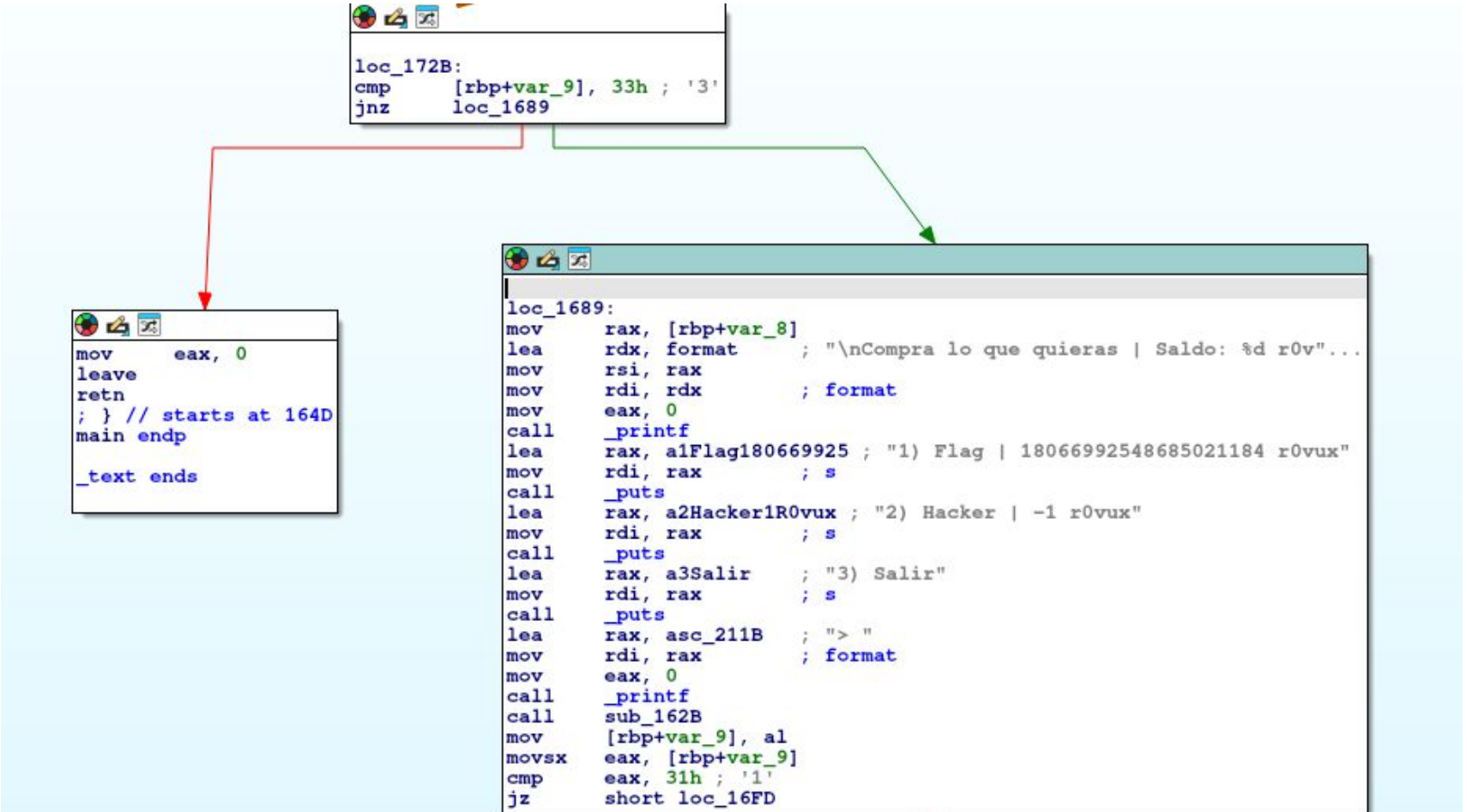
Parchear Binarios

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

¿Qué es parchear un binario?

Parchear un binario consiste en modificar las instrucciones del programa para lograr un cambio en su comportamiento



Universidad Rey Juan Carlos

Assembly x86/x64 101

Para parchear un binario tenemos que bajar al assembly

Assembly x86/x64 es muy extenso, así que solo cubriremos lo más básico





Side Quest: x86-64 ASM

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

Assembly x86/x64 101

¿Qué partes tiene una instrucción en assembly?

```
MOV RAX, 0XDEADBEEF
```

- Operadores: Es lo que quieres que la instrucción haga
- Operandos: Son los datos o lugares donde actúa el operador
  - Registros (rax, rbx, rcx, …)
  - Literales (0, 1, 67, 0x55fd7a, …)
  - Direcciones ([0x55556f005044], [rax], [rax+0x20], …)

Universidad Rey Juan Carlos

Assembly x86/x64 101 Registros

¿Qué es rax, rbx, rcx, …?

Los registros son celdas de memoria en el procesador

En x64 cada registro puede almacenar 8 bytes

Registros de propósito general:
rax, rbx, rcx, rdx, rdi, rsi, rbp, rsp, r8, r9, r10, r11, r12, r13, r14, r15

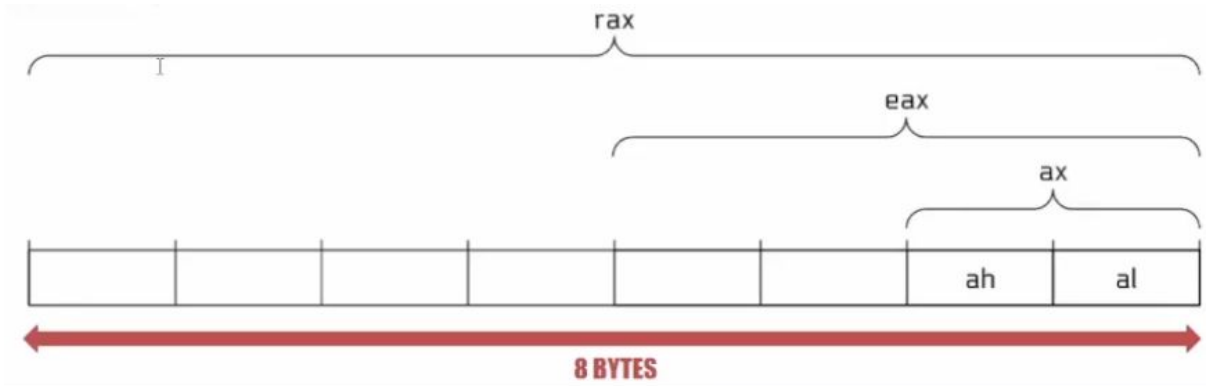
RIP:
El *rip* es un registro especial, este registro apunta a la próxima instrucción que va a ejecutar el procesador

Universidad Rey Juan Carlos

Assembly x86/x64 101 Registros

A veces veréis estos mismos registros pero con otro nombre

Dependiendo de qué nombre se use nos estaremos refiriendo a una parte específica del registro u otra



name	64-bit	32-bit	16-bit	8-bit
rax	rax	eax	ax	ah / al
rbx	rbx	ebx	bx	bh / bl
rcx	rcx	ecx	cx	ch / cl
rdx	rdx	edx	dx	dh / dl
rsi	rsi	esi	si	sil
rdi	rdi	edi	di	dil
rbp	rbp	ebp	bp	bpl
rsp	rsp	esp	sp	spl
r8	r8	r8d	r8w	r8b
r9	r9	r9d	r9w	r9b
r10	r10	r10d	r10w	r10b
r11	r11	r11d	r11w	r11b
r12	r12	r12d	r12w	r12b
r13	r13	r13d	r13w	r13b
r14	r14	r14d	r14w	r14b
r15	r15	r15d	r15w	r15b

Universidad Rey Juan Carlos

Assembly x86/x64 101 Instrucciones

Para no hacer la clase muy extensa he preparado una cheatsheet de las instrucciones más comunes y básicas de x86-64

Seguramente al reversear veréis instrucciones no mencionadas aquí

Si os es necesario entender una instrucción que no aparece en la cheatsheet, o simplemente tenéis curiosidad, recordad que google es (y va a ser) vuestro amigo (Tambien nos podeis preguntar a nosotros)

https://drive.google.com/file/d/1oKeeo2Pe49tmv6Gc7KYcxaYTb9bPqukY/view?usp=sharing

Universidad Rey Juan Carlos

Calling Convention

Cuando llamamos a una función los argumentos se tienen que pasar de alguna forma

Existe una *Calling Convention* que indica en que registros deben colocarse los argumentos al llamar a una función

El Linux x86-64 es la siguiente:

***RDI**:* Primer Argumento
***RSI**:* Segundo Argumento
***RDX**:* Tercer Argumento
***RCX**:* Cuarto Argumento
***R8**:* Quinto Argumento
***R9**:* Sexto Argumento

***RAX**:* Valor de Retorno

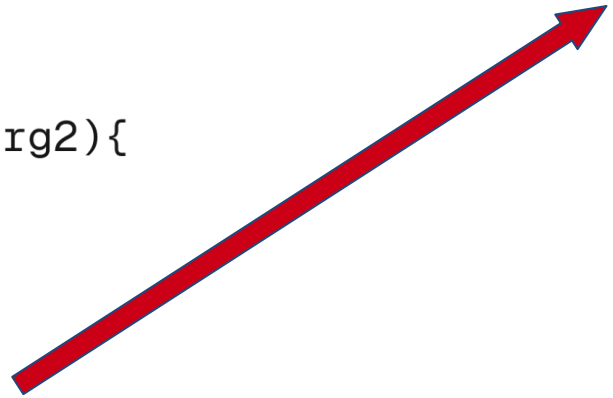
Universidad Rey Juan Carlos

Calling Convention: Ejemplo

Para visualizarlo pondré un ejemplo:

```c
int ejemplo(int arg1, int arg2){
  return arg1+arg2;
}

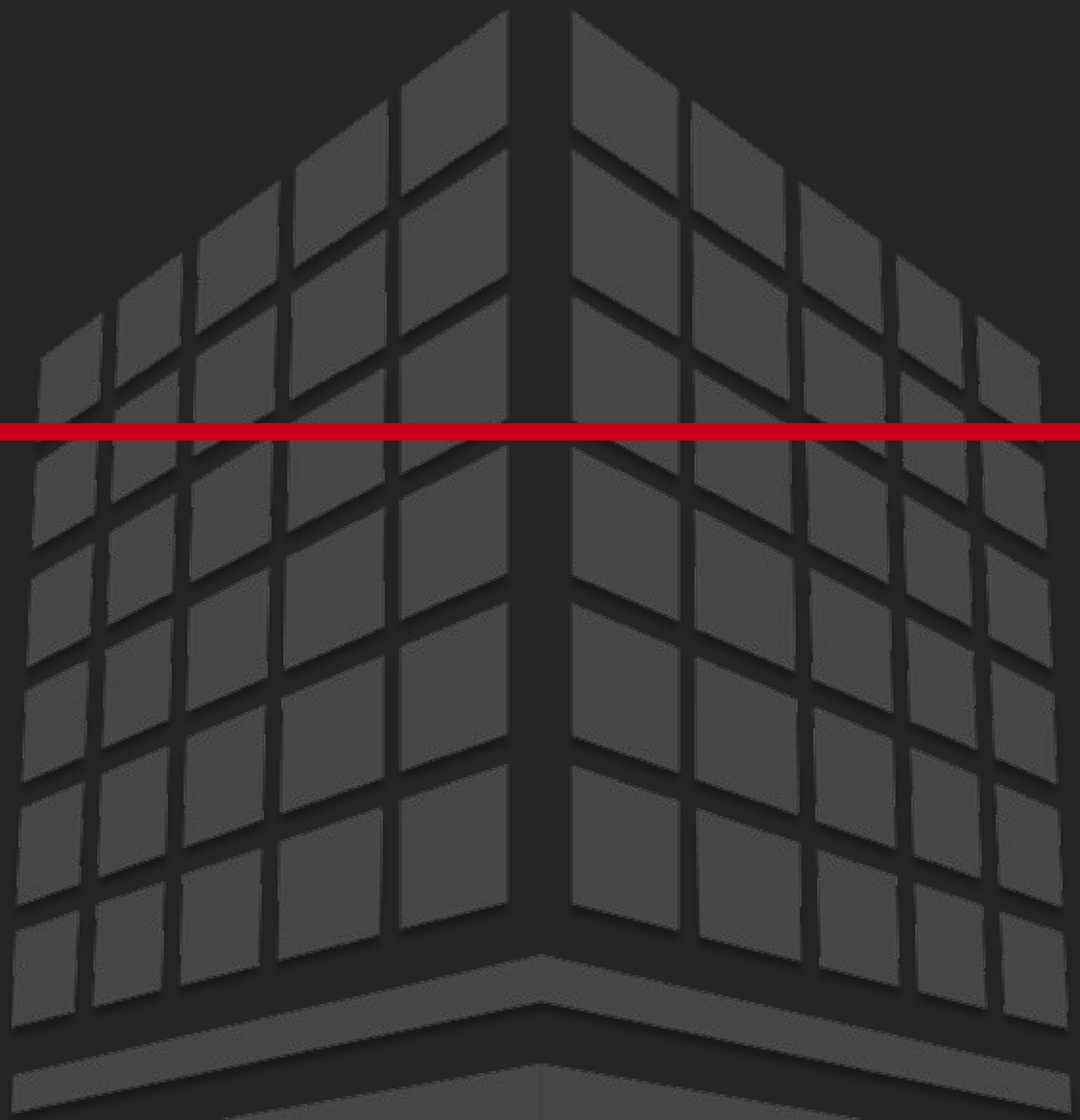
int main(){
  int result = ejemplo(43, 24);
  return 0;
}
```



#MAIN
RDI = 43
RSI = 24
CALL ejemplo
result = RAX

#EJEMPLO
RAX = RDI+RSI
RET

ret



Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

Parchear Binarios: Ejemplo práctico

Después de esta side quest vamos a lo divertido :)

Os pongo un ejemplo dónde conviene parchear el binario

Fijaos en este código:

1. Genera un número aleatorio del 0 al 1000
2. Compara si es mayor a 999

Sólo vamos a ganar con una probabilidad de 0.1%

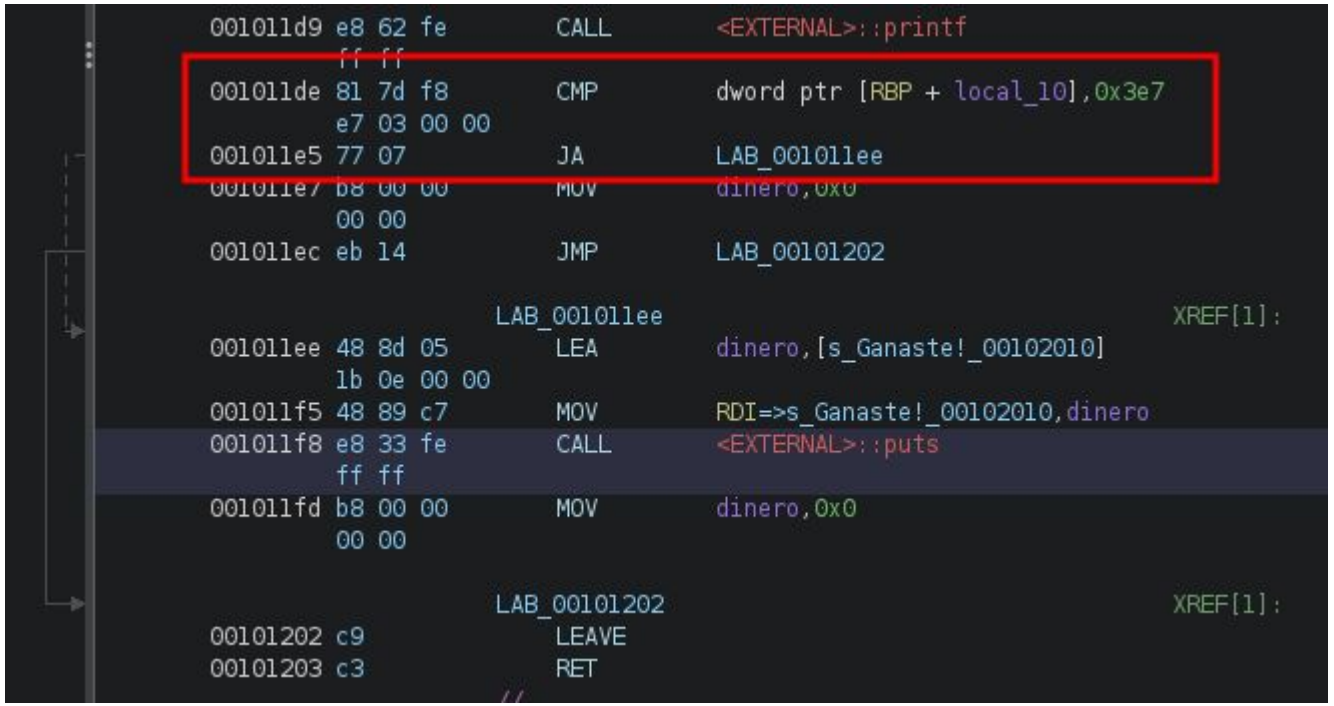
Decompile: main - (patch)

```
1
2  undefined8 main(void)
3
4  {
5    int rand_num;
6    time_t seed;
7    uint dinero;
8
9    seed = time((time_t *)0x0);
10   srand((uint)seed);
11   rand_num = rand();
12   dinero = rand_num % 1000 + 1;
13   printf("Tienes %d$\n",(ulong)dinero);
14   if (999 < dinero) {
15     puts("Ganaste!");
16   }
17   return 0;
18 }
19
```

Universidad Rey Juan Carlos

Parchear Binarios: Ejemplo práctico

Vamos a mirar el disassembler



Compara un valor en memoria con 0x3e7 (999)

Si es mayor salta a printear “Ganaste!”

Universidad Rey Juan Carlos

Parchear Binarios: Ejemplo práctico

¿Qué podemos hacer?

Podemos cambiar JA (Jump Above) por un JMP (Salto incondicional) para que siempre llegue a la ruta ganadora

```
001011d9 e8 62 fe ff ff    CALL    <EXTERNAL>::printf
001011de 81 7d f8 e7 03 00 00  CMP   dword ptr [RBP + local_10],0x3e7
001011e5 eb 07             JMP     LAB_001011ee
001011e7 b8 00 00 00 00    MOV     dinero,0x0
001011ec eb 14             JMP     LAB_00101202

                       LAB_001011ee                                XREF[1]:
001011ee 48 8d 05 1b 0e 00 00  LEA   dinero,[s_Ganaste!_00102010]
001011f5 48 89 c7          MOV     RDI=>s_Ganaste!_00102010,dinero
001011f8 e8 33 fe ff ff    CALL    <EXTERNAL>::puts
001011fd b8 00 00 00 00    MOV     dinero,0x0

                       LAB_00101202                                XREF[1]:
00101202 c9                LEAVE
00101203 c3                RET
```

Decompile: main - (patch)

```
1
2 undefined8 main(void)
3
4 {
5   int rand_num;
6   time_t seed;
7
8   seed = time((time_t *)0x0);
9   srand((uint)seed);
10  rand_num = rand();
11  printf("Tienes %d$\n",(ulong)(rand_num % 1000 + 1));
12  puts("Ganaste!");
13  return 0;
14 }
15
```

Universidad Rey Juan Carlos

Parchear Binarios: Ejemplo práctico

Ahora si guardamos el binario parcheado y lo ejecutamos ganaremos siempre!

```
zereza@kali CursoCTF/tmp » ./patched
Tienes 665$
Ganaste!
zereza@kali CursoCTF/tmp » ./patched
Tienes 4$
Ganaste!
zereza@kali CursoCTF/tmp » ./patched
Tienes 706$
Ganaste!
zereza@kali CursoCTF/tmp » ./patched
Tienes 706$
Ganaste!
zereza@kali CursoCTF/tmp » ./patched
Tienes 610$
Ganaste!
zereza@kali CursoCTF/tmp » ./patched
Tienes 610$
Ganaste!
zereza@kali CursoCTF/tmp »
```

En la cheatsheet de Ghidra se os explica cómo parchear el binario y guardarlo

https://docs.google.com/document/d/1GBogJ599b5UOGT3F_RzV8XdS4BUXvXk-xCBJjpB37Ag/edit?usp=sharing

Análisis Dinámico



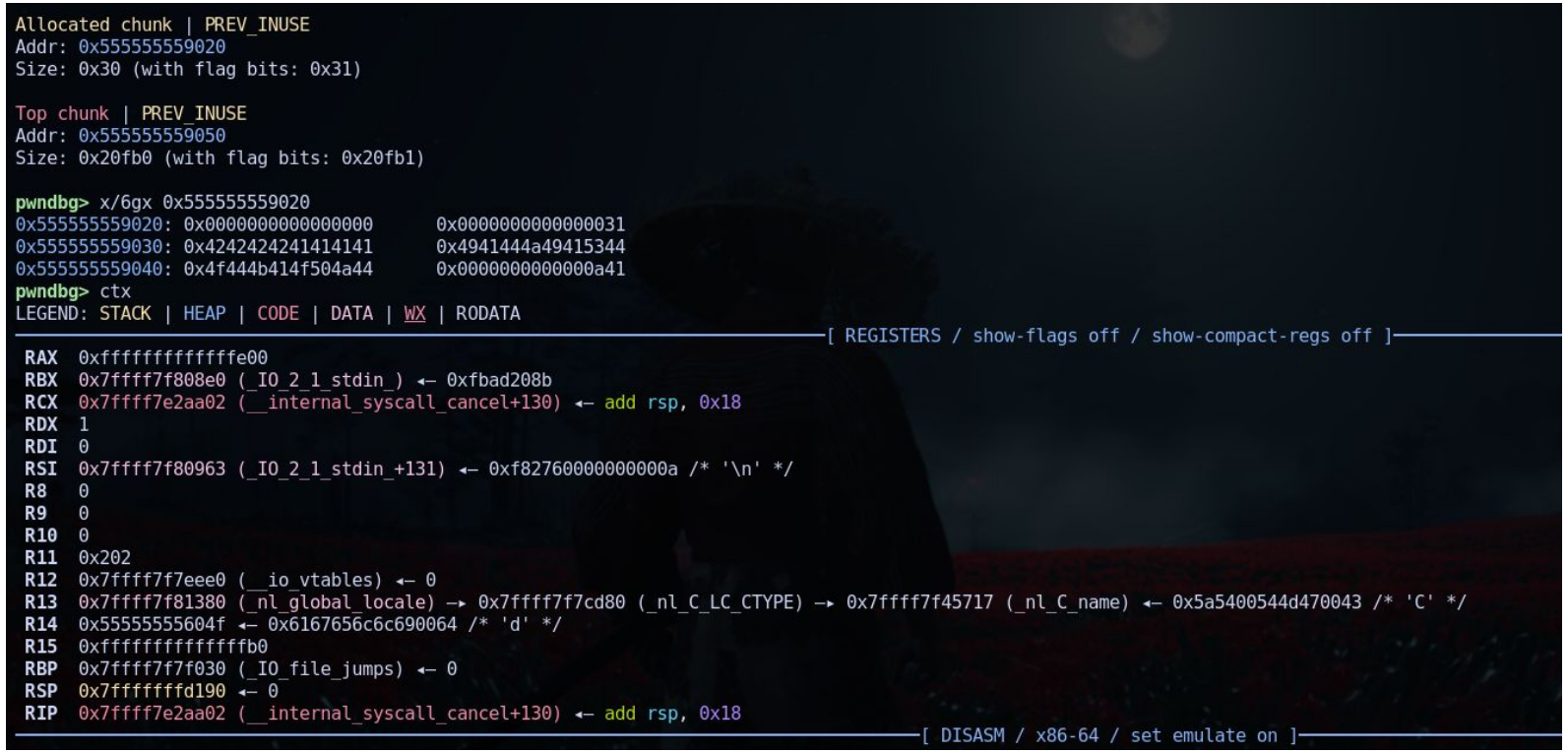
Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

Análisis Dinámico

¿Qué es el Análisis Dinámico?

El Análisis Dinámico (al contrario que el estático) se basa en analizar el programa en runtime



Universidad Rey Juan Carlos

Herramientas

Herramientas para tracear el programa

ltrace Tracea llamadas a librerías
strace Tracea llamadas al systema (*syscalls)*

Ejecutar el programa con diferentes inputs y ver como cambian las llamadas puede ayudarte

- ltrace con input A
- ltrace con input B
- Ver si puedes revertsear el algoritmo a partir del input y output

Universidad Rey Juan Carlos

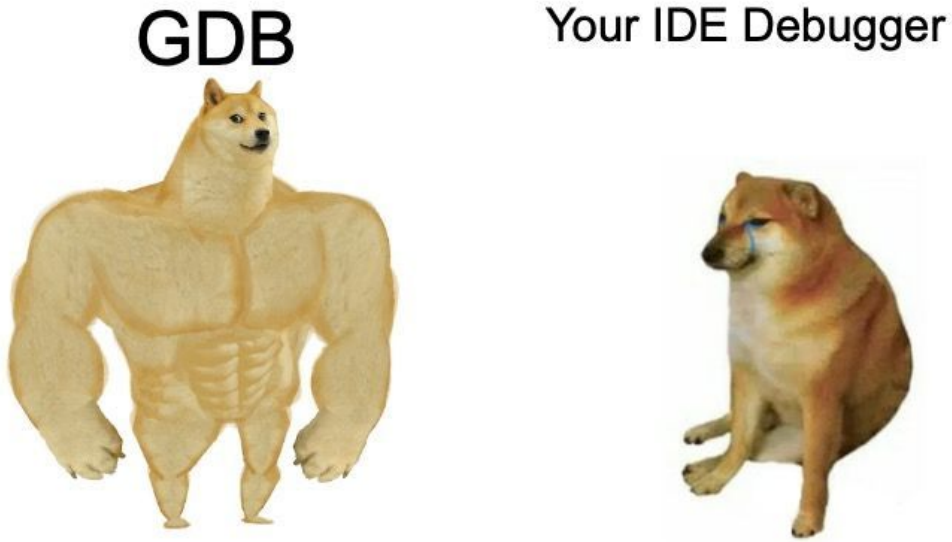
GDB

GDB (GNU Debugger) es una herramienta de debugging

(y va a convertirse en vuestro mejor amigo)

Para cualquier momento en la ejecución del binario podemos:

- Ver el valor de los registros
- Modificar los registros
- Ver la memoria
- Modificar la memoria
- Modificar el flujo de ejecución



Universidad Rey Juan Carlos

GDB

Para debuggear un binario podemos hacerlo de 2 maneras:

1. Ejecutarlo con: *gdb <binairo>*
2. Ejecutar el binario y attachearnos con: *gdb --pid <pid>*

Os hemos preparado una cheatsheet de comandos esenciales para gdb

Así podéis consultarla mientras hacéis los retos

https://drive.google.com/file/d/1HZx1PiGYuFn_LOCQ72pKnIKZCZzLDDes/view?usp=sharing

Universidad Rey Juan Carlos

Plugins GDB

Casi nadie usa GDB a pelo, lo usan con plugins

Los plugins de GDB son extensiones que amplían las capacidades de GDB

Ejemplos son: *pwndbg, gef, peda…*

Os dejo una guía para instalar *pwndbg*:

https://pwndbg.re/stable/setup/

FIN

Esperamos que os haya gustado la clase!

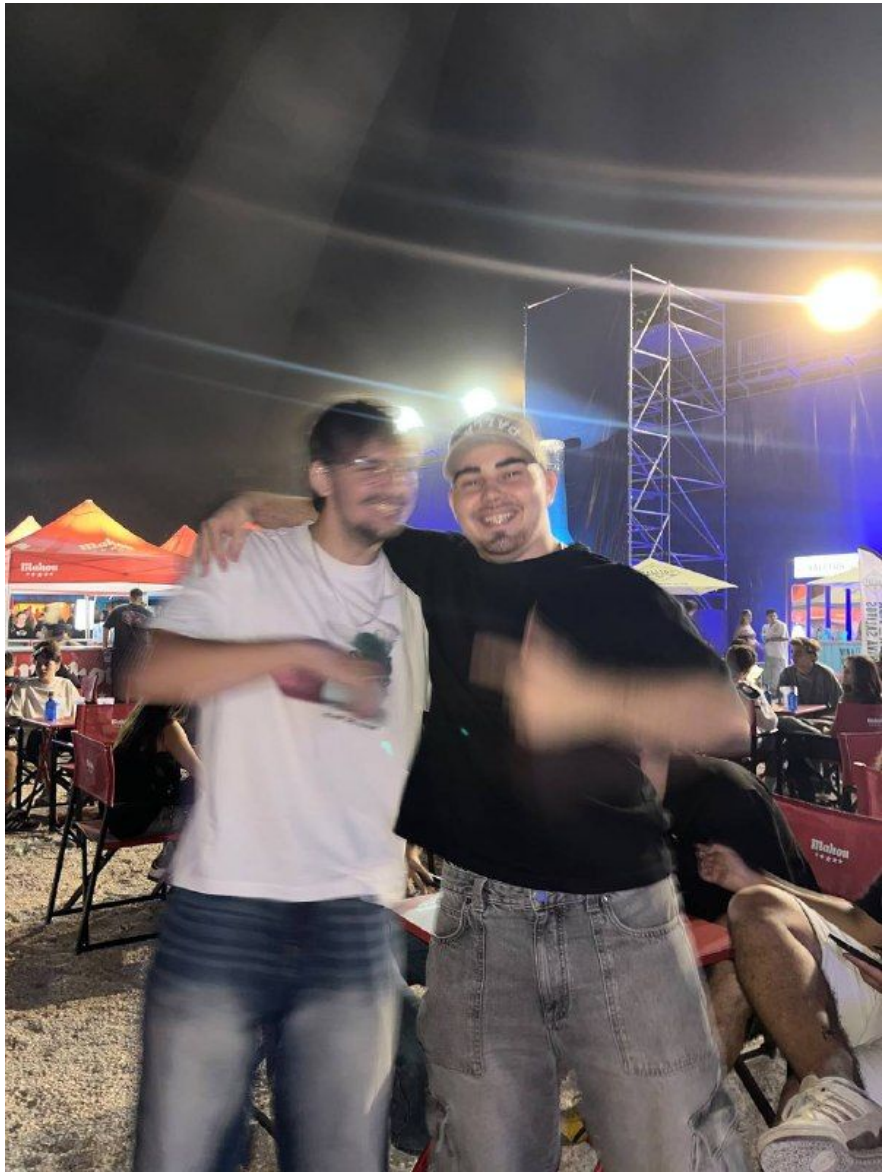
Ahora es momento de hacer los retos

Esta es la parte más importante de la clase

Creemos que solo se aprende haciendo, frustrandose y aprendiendo de los errores

Mucho ánimo con los retos!

Cualquier cosa estamos para vosotros



Lets do challenges



Universidad
Rey Juan Carlos